

CUESTIONARIO 1: Building Blocks

1. Which of the following are legal entry point methods that can be run from the command line? (Choose all that apply.)
- A. `private static void main(String[] args)`
 - B. `public static final main(String[] args)`
 - C. `public void main(String[] args)`
 - D. `public static final void main(String[] args)`
 - E. `public static void main(String[] args)`
 - F. `public static main(String[] args)`

Mi justificación
X A no es válido ser porque private porque una clase primaria no puede ser private, solo una clase dentro de una clase. X B no es válido porque void es obligatorio ya que es el valor de retorno X C no es válido porque static es necesario ya que no instanciamos main cuando ejecutamos el programa ✓ D es válido. La keyword final no afecta ya que main se declara solo una vez. ✓ E es válido ya que es la forma estándar X F no es válido por lo mismo que B

1. **D, E.** La opción E es la firma canónica del método `main()`. Debes memorizarla. La opción D es una forma alternativa que incluye el `final`, aunque es redundante. La opción A es incorrecta porque el método `main()` debe ser `public`. Las opciones B y F son incorrectas porque el método `main()` debe tener un tipo de retorno `void`. La opción C es incorrecta porque el método `main()` debe ser `static`.

2. Which answer options represent the order in which the following statements can be assembled into a program that will compile successfully? (Choose all that apply.)

X: `class Rabbit {}`

Y: `import java.util.*;`

Z: `package animals;`

A. X, Y, Z

B. Y, Z, X

C. Z, Y, X

D. Y, X

E. Z, X

F. X, Z

G. None of the above

Mi justificación
Mientras se pongan en orden Z: Nombre del paquete Y: Librerías a importar X: Nombre de la clase No hay problema, aún si se omite alguno de los 3, así que C, D y E son las únicas que cumplen con esto.

2. C, D, E. Las declaraciones `package` e `import` son ambas opcionales. Si ambas están presentes, el orden debe ser **package** → **import** → **class**. La opción A es incorrecta porque `class` aparece antes de `package` e `import`. La opción B es incorrecta porque `import` aparece antes de `package`. La opción F es incorrecta porque `class` aparece antes de `package`.

3. Which of the following are true? (Choose all that apply.)

```
public class Bunny {  
    public static void main(String[] x) {  
        Bunny bun = new Bunny();  
    } }  
}
```

- A. Bunny is a class.
- B. bun is a class.
- C. main is a class.
- D. Bunny is a reference to an object.
- E. bun is a reference to an object.
- F. main is a reference to an object.
- G. The main() method doesn't run because the parameter name is incorrect.

Mi justificación
✓ A es true ya que la primera línea indica el nombre de la clase X B es false ya que bun es una variable de referencia de un objeto X C es false ya que main es un método, no una clase X D es false ya que Bunny es la clase, no un objeto (instancia de Bunny) ✓ E es true por lo mismo que en B X F es false ya que main es un método, no una instancia de alguna clase X G es falsa ya que no hay problema si se ponen los corchetes al tipo o a la variable.

3. A, E. Bunny es una clase, como se puede ver en la declaración: public class Bunny. La variable bun es una **referencia a un objeto**. El método main() es el punto de entrada estándar de un programa. La opción G es incorrecta porque **el tipo del parámetro es lo que importa, no el nombre del parámetro**.

4. Which of the following are valid Java identifiers? (Choose all that apply.)

- A. `_`
- B. `_helloWorld$`
- C. `true`
- D. `java.lang`
- E. `Public`
- F. `1980_s`
- G. `_Q2_`

Mi justificación
X A es false ya que no puedes nombrar una variable con un solo guión bajo ✓ B es true ya que no hay problema con ponerle guión bajo siempre y cuando no este solo X C es false ya que true es palabra reservada X D es false ya que no puedes poner puntos en las variables X E es false ya que Public es palabra reservada X F es false ya que no puedes ponerle nombre de variable iniciando con número ✓ G es true por lo mismo que B

4. **B, E, G.** La opción A es incorrecta porque un guion bajo (`_`) por sí solo no está permitido. La opción C no es un identificador válido porque `true` es una palabra reservada de Java. La opción D no es válida porque un punto (`.`) no está permitido en los identificadores. La opción F no es válida porque el primer carácter no es una letra, un signo de dólar (\$) ni un guion bajo (`_`). Las opciones B, E y G son válidas porque contienen únicamente caracteres permitidos.

5. Which statements about the following program are correct? (Choose all that apply.)

```
2: public class Bear {
3:     private Bear pandaBear;
4:     private void roar(Bear b) {
5:         System.out.println("Roar!");
6:         pandaBear = b;
7:     }
8:     public static void main(String[] args) {
9:         Bear brownBear = new Bear();
10:        Bear polarBear = new Bear();
11:        brownBear.roar(polarBear);
12:        polarBear = null;
13:        brownBear = null;
14:        System.gc(); } }
```

- A. The object created on line 9 is eligible for garbage collection after line 13.
- B. The object created on line 9 is eligible for garbage collection after line 14.
- C. The object created on line 10 is eligible for garbage collection after line 12.
- D. The object created on line 10 is eligible for garbage collection after line 13.
- E. Garbage collection is guaranteed to run.
- F. Garbage collection might or might not run.
- G. The code does not compile.

Mi justificación
✓ A es true ya que Bear brownBear deja de ser accesible X B es false ya que se vuelve candidato desde la línea 13 X C es false ya que aunque Bear polarBear sea null, todavía está siendo apuntado por Bear brownBear mediante el método roar ✓ D es true ya que aquí ya se elimina la referencia del método roar antes mencionado X E es false ya que el gc no siempre se ejecuta, ni siquiera con System.gc() ✓ F es true por lo mismo que en E X G es false ya que no hay errores en el código

5. A, D, F. El *garbage collection* nunca está garantizado de ejecutarse, por lo que la opción F es correcta y la opción E es incorrecta. Además, la clase compila y se ejecuta sin problemas, por lo que la opción G es incorrecta. El objeto Bear creado en la línea 9 es accesible hasta la línea 13 mediante la variable de referencia brownBear, por lo que la opción A es correcta. El objeto Bear creado en la línea 10 es accesible tanto mediante la referencia polarBear como mediante la referencia brownBear.pandaBear. Después de la línea 12, el objeto todavía es accesible mediante brownBear.pandaBear. Sin embargo, después de la línea 13, ya no es accesible porque brownBear deja de estar accesible, por lo que la opción D es la respuesta final.

6. Assuming the following class compiles, how many variables defined in the class or method are in scope on the line marked on line 14?

```
1: public class Camel {
2:     { int hairs = 3_000_0; }
3:     long water, air=2;
4:     boolean twoHumps = true;
5:     public void spit(float distance) {
6:         var path = "";

7:         { double teeth = 32 + distance++; }
8:         while(water > 0) {
9:             int age = twoHumps ? 1 : 2;
10:            short i=-1;
11:            for(i=0; i<10; i++) {
12:                var Private = 2;
13:            }
14:            // SCOPE
15:        }
16:    }
17: }
```

- A. 2
B. 3
C. 4
D. 5
E. 6
F. 7
G. None of the above

Mi justificación

Son 4 variables, vamos una por una:

- Las variables water, air y twoHumps son variables locales de la clase pero están en el nivel más alto, accesibles dentro de toda la clase.
- La variable path está dentro de un método, pero el scope está dentro del mismo método así que si es accesible.
- La variable teeth no es accesible ya que está dentro de su propio scope por las llaves que tiene al principio y final de la línea de código.
- Las variables age, i, la otra i de la iteración y Private están dentro del scope del while. Y el scope que nos indica está fuera del while.

6. F. Para resolver este problema, debes seguir las llaves {} y observar cuándo las variables entran y salen de su **ámbito (scope)**. Las variables de las líneas 2 y 7 solo están dentro del ámbito de un bloque de una sola línea. La variable de la línea 12 solo está dentro del ámbito del ciclo for. Ninguna de estas variables está dentro del ámbito en la línea 14.

En cambio, las tres variables de instancia de las líneas 3 y 4 están disponibles en todos los métodos de instancia. Además, las variables de las líneas 6, 9 y 10 están disponibles porque tanto el método como el ciclo while siguen dentro de su ámbito. En total, hay **7 variables**, por lo que la opción correcta es **F**.

7. Which are true about this code? (Choose all that apply.)

```
public class KitchenSink {  
    private int numForks;  
  
    public static void main(String[] args) {  
        int numKnives;  
        System.out.print("""  
            "# forks = " + numForks +  
            " # knives = " + numKnives +  
            # cups = 0""");  
    }  
}
```

- A. The output includes: # forks = 0.
- B. The output includes: # knives = 0.
- C. The output includes: # cups = 0.
- D. The output includes a blank line.
- E. The output includes one or more lines that begin with whitespace.
- F. The code does not compile.

Mi justificación
El print es un bloque de texto, y este nunca se cierra hasta el final, por lo cual las supuestas concatenaciones + numForks y numKnives imprimirán tal cual el texto puro, no los valores de las variables. Así, solo # cups = 0 se imprimirá ya que está dentro del bloque de texto. Y el bloque tendrá múltiples espacios iniciales ya que presentan tabs las tres líneas.

7. C, E. Lo primero que debes reconocer es que esto es un **bloque de texto (text block)** y que el código dentro de `"""` es simplemente texto. Las opciones A y B son incorrectas porque las variables `numForks` y `numKnives` no se utilizan. Esto es conveniente porque `numKnives` no está inicializada y el código no compilaría si se hiciera referencia a ella. La opción C es correcta porque coincide con el texto. La opción D es incorrecta porque el bloque de texto no tiene una línea en blanco al final. Finalmente, la opción E también es correcta porque `" # knives"` está indentado.

8. Which of the following code snippets about var compile without issue when used in a method? (Choose all that apply.)

- A. `var spring = null;`
- B. `var fall = "leaves";`
- C. `var evening = 2; evening = null;`
- D. `var night = Integer.valueOf(3);`
- E. `var day = 1/0;`
- F. `var winter = 12, cold;`
- G. `var fall = 2, autumn = 2;`
- H. `var morning = ""; morning = null;`

Mi justificación
X A es inválido ya que var no puede ser null ya que debe tener un valor para inferir su tipo, y null no permite esto. ✓ B es válido ya que "leaves" puede inferirse sin problemas que es del tipo String X C es inválido ya que el punto y coma indica otra instrucción, y evening no tiene var ni keyword de tipo así que es inválido ✓ D es válido ya que aquí ya se elimina la referencia del método <code>valueOf</code> antes mencionado X E es inválido ya que <code>1/0</code> genera una excepción aritmética. X F es inválido ya que no puedes declarar múltiples var en la misma instrucción X G es inválido por lo mismo que F X H es inválido por lo mismo que C

8. B, D, E, H. Una variable var no puede inicializarse con un valor null sin especificar un tipo, pero posteriormente sí puede asignársele un valor null si el tipo subyacente no es un primitivo. Por estas razones, la opción H es correcta, mientras que las opciones A y C son incorrectas.

Las opciones B y D son correctas porque los tipos subyacentes son String e Integer, respectivamente. La opción E es correcta porque se trata de una expresión numérica válida. Es posible que sepas que dividir entre cero produce una excepción en tiempo de ejecución, pero la pregunta solo se refiere a si el código **compila**.

Finalmente, las opciones F y G son incorrectas porque var no puede utilizarse en una asignación de múltiples variables.

9. Which of the following are correct? (Choose all that apply.)
- A. An instance variable of type `float` defaults to `0`.
 - B. An instance variable of type `char` defaults to `null`.
 - C. A local variable of type `double` defaults to `0.0`.
 - D. A local variable of type `int` defaults to `null`.
 - E. A class variable of type `String` defaults to `null`.
 - F. A class variable of type `String` defaults to the empty string `""`.
 - G. None of the above.

Mi justificación
X A es false ya que de una variable de instancia de tipo <code>float</code> valdría <code>0.0</code> X B es false ya que el valor default de <code>char</code> de instancia es <code>0</code>. X C es false ya que las variables locales no tienen valor default. X D es false ya que las variables locales no tienen valor default. X E es false ya que el valor default de <code>String</code> es <code>""</code> ✓ F es true ya que el valor default de <code>String</code> es <code>""</code> X G es false ya que F es true.

9. E. Las opciones C y D son incorrectas porque las variables locales **no tienen valores predeterminados**. La opción A es incorrecta porque un `float` debe tener un punto decimal. La opción B es incorrecta porque los tipos primitivos no tienen `null` como valor predeterminado. La opción E es correcta y la opción F es incorrecta porque los tipos de referencia en las variables de clase tienen `null` como valor predeterminado.

10. Which of the following expressions, when inserted independently into the blank line, allow the code to compile? (Choose all that apply.)

```
public void printMagicData() {  
    var magic = _____;  
    System.out.println(magic);  
}
```

- A. 3_1
- B. 1_329_.0
- C. 3_13.0_
- D. 5_291._2
- E. 2_234.0_0
- F. 9___6
- G. _1_3_5_0

Mi justificación
Siguiendo las reglas de definir un numérico: 1. No se puede poner guión al inicio o al final del número 2. No se puede poner guión justo antes o después del punto decimal A, E y F cumplen estas reglas, los demás rompen una o ambas reglas. 10. A, E, F. Se puede colocar un guion bajo (_) en cualquier literal numérico, siempre que no esté al principio, al final ni junto a un punto decimal (.). Incluso se pueden colocar guiones bajos uno junto a otro. Por estas razones, las opciones A, E y F son correctas. Las opciones B y D son incorrectas porque el guion bajo (_) está junto a un punto decimal (.). Las opciones C y G son incorrectas porque no se puede colocar un guion bajo (_) al principio ni al final del literal.

11. Given the following two class files, what is the maximum number of imports that can be removed and have the code still compile?

// Water.java

```
package aquarium;  
public class Water { }
```

// Tank.java

```
package aquarium;  
import java.lang.*;  
import java.lang.System;  
import aquarium.Water;  
import aquarium.*;  
public class Tank {  
    public void print(Water water) {  
        System.out.println(water); } }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4
- F. Does not compile

Mi justificación
La primera y segunda línea no son necesarias de importar porque lang viene por default ya que es el paquete de los wrapper classes. La tercera y cuarta línea tampoco son necesarias porque la clase Tank pertenece al mismo paquete que la clase Water, así que no requiere poner import.

11. E. Los dos primeros import se pueden eliminar porque java.lang se importa automáticamente. Los dos import siguientes también se pueden eliminar porque Tank y Water están en el mismo paquete, por lo que la opción correcta es E.

Si Tank y Water estuvieran en paquetes diferentes, exactamente uno de estos dos import podría eliminarse. En ese caso, la respuesta sería la opción D.

12. Which statements about the following class are correct? (Choose all that apply.)

```
1: public class ClownFish {  
2:     int gills = 0, double weight=2;  
3:     { int fins = gills; }  
4:     void print(int length = 3) {  
5:         System.out.println(gills);  
6:         System.out.println(weight);  
7:         System.out.println(fins);  
8:         System.out.println(length);  
9:     } }
```

- A. Line 2 generates a compiler error.
- B. Line 3 generates a compiler error.
- C. Line 4 generates a compiler error.
- D. Line 7 generates a compiler error.
- E. The code prints 0.
- F. The code prints 2.0.
- G. The code prints 2.
- H. The code prints 3.

Mi justificación

✓ A es true ya que la coma es para declarar variables del mismo tipo en una misma línea de código
X B es false ya que no hay error, solo que fins existirá dentro de sus llaves nada más.
✓ C es true ya que no puedes declarar variables con valor dentro de una función.
✓ D es true ya que como se mencionó para B, fins solo existe dentro de sus llaves, por lo que no es posible imprimirla en el scope de void print().
X E es false ya que print no pudo definirse por el parámetro mal escrito.
X F es false ya que print no pudo definirse por el parámetro mal escrito.
X G es false ya que print no pudo definirse por el parámetro mal escrito.
X H es false ya que print no pudo definirse por el parámetro mal escrito.

12. **A, C, D.** La línea 2 no compila porque solo se debe especificar un tipo, por lo que la opción A es correcta. La línea 3 compila sin problemas porque declara una variable local dentro de un **inicializador de instancia** que nunca se utiliza.

La línea 4 no compila porque Java no permite establecer valores predeterminados para los parámetros de un método, por lo que la opción C es correcta. Finalmente, la línea 7 no compila porque fins está dentro del ámbito (*scope*) y solo es accesible dentro del inicializador de instancia de la línea 3, por lo que la opción D es correcta.

13. Given the following classes, which of the following snippets can independently be inserted in place of INSERT IMPORTS HERE and have the code compile? (Choose all that apply.)

```
package aquarium;
public class Water {
    boolean salty = false;
}
```

```
package aquarium.jellies;
public class Water {
    boolean salty = true;
}
```

```
package employee;
INSERT IMPORTS HERE
public class WaterFiller {
    Water water;
}
```

- A. `import aquarium.*;`
- B. `import aquarium.Water;`
`import aquarium.jellies.*;`
- C. `import aquarium.*;`
`import aquarium.jellies.Water;`
- D. `import aquarium.*;`
`import aquarium.jellies.*;`
- E. `import aquarium.Water;`
`import aquarium.jellies.Water;`
- F. None of these imports can make the code compile.

Mi justificación

Ninguna combinación de importación de AMBOS paquetes funcionaría ya que `aquarium` y `aquarium.jellies` tienen la misma `public class Water`, lo cual genera una ambigüedad en el caso de que se importen ambas. Solo funcionaría si se importa o `aquarium` o `aquarium.jellies`, es decir solo uno.

Por ello, solo la opción A es válida.

13. **A, B, C.** La opción A es correcta porque importa todas las clases del paquete `aquarium`, incluyendo `aquarium.Water`. Las opciones B y C son correctas porque importan `Water` mediante su nombre de clase. Como importar mediante el nombre de la clase tiene **precedencia sobre los comodines (*)**, estas opciones compilan.

La opción D es incorrecta porque Java no sabe cuál de las dos clases `Water` importadas mediante comodines debe utilizar. La opción E es incorrecta porque no se puede especificar el mismo nombre de clase en dos `import`.

14. Which of the following statements about the code snippet are true? (Choose all that apply.)

```
3: short numPets = 5L;  
4: int numGrains = 2.0;  
5: String name = "Scruffy";  
6: int d = numPets.length();  
7: int e = numGrains.length;  
8: int f = name.length();
```

- A. Line 3 generates a compiler error.
- B. Line 4 generates a compiler error.
- C. Line 5 generates a compiler error.
- D. Line 6 generates a compiler error.
- E. Line 7 generates a compiler error.
- F. Line 8 generates a compiler error.

Mi justificación
✓ A es true ya que no puedes convertir 5L a short sin casteo explícito ✓ B es true ya que no puede convertir 2.0 a int sin casteo explícito X C es false ya que no hay problemas con la declaración de un String ✓ D es true ya que numPets es una variable primitiva así que no tiene atributos ni métodos, por lo tanto length() no es valido. ✓ E es true ya que numGrains es una variable primitiva así que no tiene atributos ni métodos, por lo tanto length no es valido (length es solo para arreglos). X F es false ya que String es un objeto que tiene como método length(), así que si se puede dar valor de name.length().

14. **A, B, D, E.** La línea 3 no compila porque el sufijo L hace que el literal sea de tipo long, y un long no se puede almacenar directamente en un short, por lo que la opción A es correcta.

La línea 4 no compila porque int es un tipo entero, mientras que 2.0 es un literal de tipo double, por lo que la opción B es correcta. La línea 5 compila sin problemas.

Las líneas 6 y 7 no compilan porque numPets y numGrains son tipos primitivos, y solo se pueden llamar métodos sobre **tipos de referencia**, no sobre valores primitivos, por lo que las opciones D y E son correctas, respectivamente.

Finalmente, la línea 8 compila porque existe un método length() definido en String.

15. Which of the following statements about garbage collection are correct? (Choose all that apply.)
- A. Calling `System.gc()` is guaranteed to free up memory by destroying objects eligible for garbage collection.
 - B. Garbage collection runs on a set schedule.
 - C. Garbage collection allows the JVM to reclaim memory for other objects.
 - D. Garbage collection runs when your program has used up half the available memory.
 - E. An object may be eligible for garbage collection but never removed from the heap.
 - F. An object is eligible for garbage collection once no references to it are accessible in the program.
 - G. Marking a variable `final` means its associated object will never be garbage collected.

Mi justificación
X A es false ya que puede que no haya nada que destruir X B es false ya que no hay una condición que haga que gc se ejecute. ✓ C es true ya que ese justo es el funcionamiento del gc, liberar memoria para que otros objetos la usen. X D es false ya que no hay una condición que haga que gc se ejecute. ✓ E es true porque no siempre ser elegible para gc indica que te borren instantáneamente. ✓ F es true ya que una condición que te hace elegible para gc es que no puedas ser accedido mediante ninguna variable de referencia. X G es false ya que un objeto puede inicializarse como null y esto por definición lo haría candidato para el gc.

15. **C, E, F.** En Java, no hay ninguna garantía sobre cuándo se ejecutará el *garbage collection*. La JVM puede ignorar las llamadas a `System.gc()`. Por esta razón, las opciones A, B y D son incorrectas.

La opción C es correcta porque el propósito del *garbage collection* es **recuperar la memoria utilizada**. La opción E también es correcta: un objeto puede no ser recolectado nunca, por ejemplo, si el programa termina antes de que se ejecute el *garbage collection*.

La opción F es correcta y representa el medio principal mediante el cual los algoritmos de *garbage collection* determinan si un objeto es **elegible para ser recolectado**.

Finalmente, la opción G es incorrecta porque marcar una variable como `final` significa que es **constante dentro de su propio ámbito**. Por ejemplo, una variable local marcada como `final` será elegible para *garbage collection* después de que termine el método, suponiendo que no existan otras referencias al objeto fuera del método.

16. Which are true about this code? (Choose all that apply.)

```
var blocky = ""  
    squirrel \s  
    pigeon  \  
    termite"";  
System.out.print(blocky);
```

- A. It outputs two lines.
- B. It outputs three lines.
- C. It outputs four lines.
- D. There is one line with trailing whitespace.
- E. There are two lines with trailing whitespace.
- F. If we indented each line five characters, it would change the output.

Mi justificación
El texto final es Squirrel(espacio) Pigeon termite Lo cual son dos líneas, y ese (espacio) es un trailing whitespace

16. A, D. La opción A es correcta. Hay dos líneas: una comienza con squirrel y la otra con pigeon. Recuerda que una barra invertida (\) significa **omitir el salto de línea**.

La opción D también es correcta porque \s significa **conservar el espacio en blanco**. En un bloque de texto (*text block*), la indentación incidental se ignora, por lo que la opción F es incorrecta.

17. What lines are printed by the following program? (Choose all that apply.)

```
1: public class WaterBottle {
2:     private String brand;
3:     private boolean empty;
4:     public static float code;
5:     public static void main(String[] args) {
6:         WaterBottle wb = new WaterBottle();

7:         System.out.println("Empty = " + wb.empty);
8:         System.out.println("Brand = " + wb.brand);
9:         System.out.println("Code = " + code);
10:    } }
```

- A. Line 8 generates a compiler error.
- B. Line 9 generates a compiler error.
- C. Empty =
- D. Empty = false**
- E. Brand =
- F. Brand = null**
- G. Code = 0.0**
- H. Code = 0f

Mi justificación
X A es false ya que atributo boolean sin darle valor es false X B es false ya que atributo String sin darle valor es null X C es false ya que es false, no "" ✓ D es true X E es false ya que el valor de los objetos sin darles valor inicial es null, no "" ✓ F es true ✓ G es true ya que atributo float sin darle valor es 0.0 X H es false porque atributo float sin darle valor es 0.0, no 0f

17. **D, F, G.** El código compila y se ejecuta sin problemas, por lo que las opciones A y B son incorrectas. Un campo de tipo boolean se inicializa con false, por lo que la opción D es correcta y se imprime Empty = false.

Las referencias a objetos se inicializan con null, no con una cadena vacía (""), por lo que la opción F es correcta y se imprime Brand = null.

Finalmente, el valor predeterminado de los números de punto flotante es 0.0. Aunque los valores float pueden declararse con el sufijo f, no se imprimen con dicho sufijo. Por estas razones, la opción G es correcta y se imprime Code = 0.0.

18. Which of the following statements about var are true? (Choose all that apply.)

- A. A var can be used as a constructor parameter.
- B. The type of a var is known at compile time.
- C. A var cannot be used as an instance variable.
- D. A var can be used in a multiple variable assignment statement.
- E. The value of a var cannot change at runtime.
- F. The type of a var cannot change at runtime.
- G. The word var is a reserved word in Java.

Mi justificación
X A es false ya que los parámetros necesitan su tipo definido ✓ B es true ✓ C es true porque var solo sirve para variables locales, no atributos de instancia ni de clase X es false, cada var debe declararse en una instrucción diferente X es false, sólo no puede cambiar si es final F es true ya que el tipo de var solo se infiere una vez y no puede cambiarse después ✓ G es true

18. **B, C, F.** var no se puede utilizar para un parámetro de constructor o método, ni para una variable de instancia o de clase, por lo que la opción A es incorrecta y la opción C es correcta.

El tipo de una variable declarada con var se conoce en **tiempo de compilación**, y el tipo no puede cambiar durante la ejecución, aunque su valor sí puede cambiar durante la ejecución. Por estas razones, las opciones B y F son correctas, mientras que la opción E es incorrecta.

La opción D es incorrecta porque var no está permitido en declaraciones de múltiples variables.

Finalmente, la opción G es incorrecta porque var **no es una palabra reservada** en Java.

19. Which are true about the following code? (Choose all that apply.)

```
var num1 = Long.parseLong("100");  
var num2 = Long.valueOf("100");  
System.out.println(Long.max(num1, num2));
```

- A. The output is 100.
- B. The output is 200.
- C. The code does not compile.
- D. num1 is a primitive.
- E. num2 is a primitive.

Mi justificación
✓ A es true ya que se están comparando dos long con valor de 100, dando 100 el máximo X B es false porque el output es 100 X C es false, si compila ✓ D es true ya que num1 usa parseLong, lo cual devuelve long, no Long X E es false ya que valueOf devuelve un wrapped class Long, el cual es un objeto.

19. A, D. Las dos primeras líneas proporcionan una forma de convertir un String en un número. La primera es un primitivo long y la segunda es un objeto de referencia Long, por lo que la opción D es una de las respuestas.

El código es correcto y el valor máximo es **100**, por lo que la opción A es correcta.

```

7:    }
8:    public static void main(String []time) {
9:        var p = new PoliceBox();
10:       var q = new PoliceBox();
11:       p.color = "green";
12:       p.age = 1400;
13:       p = q;
14:       System.out.println("Q1="+q.color);
15:       System.out.println("Q2="+q.age);
16:       System.out.println("P1="+p.color);
17:       System.out.println("P2="+p.age);
18: } }

```

- A. It prints Q1=blue.
- B. It prints Q2=1200.
- C. It prints P1=null.**
- D. It prints P2=1400.
- E. Line 4 does not compile.
- F. Line 12 does not compile.
- G. Line 13 does not compile.
- H. None of the above.

20. Which statements about the following class are correct? (Choose all that apply.)

```

1: public class PoliceBox {
2:     String color;
3:     long age;
4:     public void PoliceBox() {
5:         color = "blue";
6:         age = 1200;

```

Mi justificación

PoliceBox() no es un constructor ya que se le puso void, así que es un método, por lo tanto, crear una instancia de PoliceBox no hace que se ejecute, se tendrá que ejecutar policeBox.PoliceBox() para ejecutarlo.

X A es false ya que nunca se ejecutó q.PoliceBox() para alterar los valores sin inicializar de color ni age, así que q.color es null

X B es false ya que nunca se ejecutó q.PoliceBox() para alterar los valores sin inicializar de color ni age así que q.age es 0

✓ C es true ya que al hacer p = q, ahora p apunta al mismo PoliceBox que q, así que sí q.color es null, entonces p.color también es null

X D es false ya que al hacer p = q, ahora p apunta al mismo PoliceBox que q, así que sí q.age es 0, entonces p.age también es 0, no 1400

X E, F y G son false ya que todo compila bien

X H es false ya que C es true

20. C. Lo más importante es notar que la línea 4 **no define un constructor**, sino un método llamado PoliceBox(), ya que tiene un tipo de retorno void. Este método nunca se ejecuta durante la ejecución del programa, y color y age reciben los valores predeterminados null y 0L, respectivamente.

Las líneas 11 y 12 cambian los valores del objeto asociado con p, pero después, en la línea 13, la variable p cambia para apuntar al objeto asociado con q, que todavía tiene los valores predeterminados.

Por esta razón, el programa imprime:

Q1=null, Q2=0, P1=null, P2=0

Por lo tanto, la opción **C** es la única respuesta correcta.

21. What is the output of executing the following class?

```
1: public class Salmon {
2:     int count;
3:     { System.out.print(count+"-"); }
4:     { count++; }
5:     public Salmon() {
6:         count = 4;
7:         System.out.print(2+"-");
8:     }
9:     public static void main(String[] args) {
10:        System.out.print(7+"-");
11:        var s = new Salmon();
12:        System.out.print(s.count+"-"); } }
```

- A. 7-0-2-1-
- B. 7-0-1-
- C. 0-7-2-1-
- D. 7-0-2-4-**
- E. 0-7-1-
- F. The class does not compile because of line 3.
- G. The class does not compile because of line 4.
- H. None of the above.

Mi justificación

Salmon tiene inicializadores antes de su constructor, así que el flujo es

1. Se imprime "7-"
2. Se crea el objeto s, que es Salmon, entonces se ejecutan los inicializadores int count = 0, se imprime "0 -" y después se le suma 1, entonces count = 1
3. Se ejecuta el constructor, entonces count = 4, se imprime "2-"
4. Se ejecuta la línea 12, el cual imprime "4-"

Al final se imprimió "7-0-2-4-"

21. **D.** Comenzamos con el método main(), que imprime 7- en la línea 10. Después, en la línea 11 se crea una nueva instancia de Salmon. Esto hace que se ejecuten, en orden, los dos **inicializadores de instancia** de las líneas 3 y 4.

El valor predeterminado de una variable de instancia de tipo int es 0, por lo que lo siguiente que se imprime es 0-, y count recibe el valor 1. Después se llama al constructor. Este asigna el valor 4 a count e imprime 2-.

Finalmente, la línea 12 imprime 4-, ya que ese es el valor de count.

Juntando todo, obtenemos:

7-0-2-4-

Por lo tanto, la opción **D** es la respuesta correcta.

22. Given the following class, which of the following lines of code can independently replace INSERT CODE HERE to make the code compile? (Choose all that apply.)

```
public class Price {  
    public void admission() {  
        INSERT CODE HERE  
        System.out.print(amount);  
    }  
}
```

- A. `int Amount = 0b11;`
- B. `int amount = 9L;`
- C. `int amount = 0xE;`
- D. `int amount = 1_2.0;`
- E. `double amount = 1_0_.0;`
- F. `int amount = 0b101;`
- G. `double amount = 9_2.1_2;`
- H. `double amount = 1_2_.0_0;`

Mi justificación
X A es inválido ya que las variables primitivas no pueden iniciar con mayúscula. X B es inválido ya que la conversión de long a int requiere casteo explícito ✓ C es válido ya que es notación hexadecimal por el prefijo 0x, E es 14 en hexadecimal. X D es inválido ya que la conversión de double a int requiere casteo explícito. X E es inválido ya que no debe haber guiones bajo justo antes o después del punto decimal. ✓ F es válido ya que la variable está escrita con minúsculas y es un binario válido ✓ G es válido ya que cumple con las reglas del guión bajo en numéricos. X H es inválido ya que no debe haber guiones bajo justo antes o después del punto decimal.

22. C, F, G. Primero, 0b es el prefijo para un valor binario y 0x es el prefijo para un valor hexadecimal. Estos valores se pueden asignar a varios tipos primitivos, incluyendo int y double, por lo que las opciones C y F son correctas.

La opción A es incorrecta porque nombrar la variable Amount hará que la llamada System.out.print(amount) de la siguiente línea no compile. La opción B es incorrecta porque 9L es un valor de tipo long. Si se cambiara el tipo a long amount = 9L, entonces compilaría.

La opción D es incorrecta porque 1_2.0 es un valor de tipo double. Si se cambiara el tipo a double amount = 1_2.0, entonces compilaría.

Las opciones E y H son incorrectas porque el guion bajo (_) aparece junto al punto decimal (.), lo cual no está permitido.

Finalmente, la opción G es correcta, y tanto el uso del guion bajo como el de la asignación son válidos.

23. Which statements about the following class are true? (Choose all that apply.)

```
1: public class River {
2:     int Depth = 1;
3:     float temp = 50.0;
4:     public void flow() {
5:         for (int i = 0; i < 1; i++) {
6:             int depth = 2;
7:             depth++;
8:             temp--;
9:         }

10:         System.out.println(depth);
11:         System.out.println(temp); }
12:     public static void main(String... s) {
13:         new River().flow();
14:     } }
```

- A. Line 3 generates a compiler error.
- B. Line 6 generates a compiler error.
- C. Line 7 generates a compiler error.
- D. Line 10 generates a compiler error.**
- E. The program prints 3 on line 10.
- F. The program prints 4 on line 10.
- G. The program prints 50.0 on line 11.
- H. The program prints 49.0 on line 11.

Mi justificación

X A es false, no hay problema
X B es false, no hay problema
X C es false, no hay problema
✓ D es true ya que Depth solo existe dentro del scope de las iteraciones de for.
X E es false ya que no compila
X F es false ya que no compila
X G es false ya que no compila
X H es false ya que no compila

23. A, D. El primer error de compilación está en la línea 3. La variable temp está declarada como float, pero el valor asignado es 50.0, que es un double porque no tiene el sufijo F/f. Como un double no cabe dentro de un float sin realizar una conversión explícita, la línea 3 no compila.

Después, depth se declara dentro del ciclo for y solo tiene **ámbito (scope) dentro de este ciclo**. Por lo tanto, intentar leer su valor en la línea 10 provoca un error de compilación.

Por estas razones, las opciones **A y D** son las respuestas correctas.

Resultado final: 16/23

Calificación = 7